Fall 2026

CS-UY 2413 (Sections A and B): Design and Analysis of Algorithms

Prof. Lisa Hellerstein
lisa.hellerstein@nyu.edu

Lecture 6

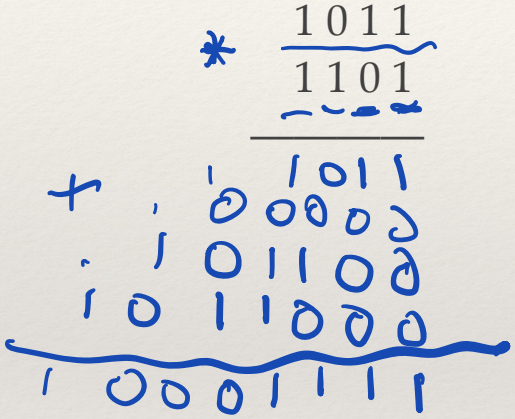
- Recommended Reading:
  - Chap. 4, Chap. 6
  - Note: we will not be covering the Master Theorem version in Chap. 4 or Strassen's algorithm for fast matrix multiplication

Multiplication of Large Integers

Multiplying large integers

- We usually consider multiplying two integers to be a basic operation (so takes constant time)
  - Computers have dedicated hardware to multiply two integers up to a given size very quickly, in parallel
- But what about multiplying two very large integers?
  - Large integers require many words to store
  - Not reasonable to consider this a basic operation
- Question: How many bit operations needed to multiply two n-bit numbers?
  - Related question: How many word operations needed to multiply two n-word numbers.

- Standard method to multiply two n-bit numbers



- Involves adding n numbers, of length between n and $2n$-1 bits
- $\theta(n^2)$ bit operations
- Result has at most $2n$ bits

- Faster method, using divide and conquer
- Karatsuba's algorithm
  - not in textbook
  - In Dasgupta *et al.*'s *Algorithms* book (section 2.1)
  - In Jeff Erickson's Algorithms book, section 1.9 (page 40) (available for download from http://jeffe.cs.illinois.edu/teaching/algorithms/#book)
  - also see Wikipedia entry
  - Similar idea used in Strassen's algorithm for matrix multiplication, covered in textbook
    - We won't cover Strassen's algorithm

Multiplying two n-bit numbers

- For multiplying two n-bit numbers
  - We assume n a power of 2
    - Can pad numbers with initial 0's to make length power of 2
    - At most increases length by 2
- Note: Can ***add*** or ***subtract*** two n-bit numbers in $\theta(n)$ bit operations, result has at most $n + 1$ bits

0110
0101

Karatsuba's algorithm

- Idea: x and y are n-bit integers, n a power of 2
  - Break x and y into two halves

$$x_L \quad x_R$$
$$y_L \quad y_R$$

$$x = 2^{n/2}x_L + x_R \qquad y = 2^{n/2}y_L + y_R$$

e.g.
$x = 0110$
$y = 1101$

$x_L = 01$ $x_R = 10$ $n = 4$
$y_L = 11$ $y_R = 01$ $n/2 = 2$

$2^{n/2}x_L = 0100$

$$xy = (2^{n/2}x_L + x_R)(2^{n/2}y_L + y_R) =$$

$$2^n x_L y_L + 2^{n/2} x_R y_L + 2^{n/2} x_L y_R + x_R y_R$$

  - Suggests recursive algorithm:
    - Recursively compute the products $x_L y_L$, $x_L y_R$, $x_R y_L$, $x_R y_R$
    - Combine results using above formula

Karatsuba's algorithm

- Recursive algorithm (first try):
  1. Break x and y into two halves: x_L and x_R, y_L and y_R
  2. Recursively compute the products $x_L y_L$, $x_L y_R$, $x_R y_L$, $x_R y_R$
  3. Compute and then return

$$2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$$

11 01 110 10

x = 0110
y = 1101

$x_L = 01$ $x_R = 10$

$y_L = 11$ $y_R = 01$

Karatsuba's algorithm

- Recursive algorithm (first try):
  1. Break x and y into two halves: x_L and x_R, y_L and y_R
  2. Recursively compute the products $x_L y_L$, $x_L y_R$, $x_R y_L$, $x_R y_R$
  3. Compute and then return
     $$2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$$
- How many bit operations in third step?
  - To obtain x_L, x_R, y_L, y_R? $\Theta(n)$
  - To add $x_L y_R$ and $x_R y_L$? $\Theta(n)$
  - To multiply by 2^n and $2^{n/2}$? $\Theta(n)$
  - To add the three terms in the expression? $\Theta(n)$

Karatsuba's algorithm

- Recursive algorithm (first try):
  1. Break x and y into two halves: x_L and x_R, y_L and y_R
  2. Recursively compute the products $x_L y_L$, $x_L y_R$, $x_R y_L$, $x_R y_R$
  3. Compute and then return
     $$2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$$
- How many bit operations in third step? $\theta(n)$ total
  - To obtain x_L, x_R, y_L, y_R? $\theta(n)$
  - To add $x_L y_R$ and $x_R y_L$? $\theta(n)$
  - To multiply by 2^n and $2^{n/2}$? $\theta(n)$ using left shift to multiply by $2^{n/2}$ and 2^n
  - To add the three terms in the expression? $\theta(n)$

Karatsuba's algorithm

- Recursive algorithm (first try):
  1. Break x and y into two halves: x_L and x_R, y_L and y_R
  2. Recursively compute the products $x_L y_L$, $x_L y_R$, $x_R y_L$, $x_R y_R$
  3. Compute and then return
     $$2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$$
- Let $B(n)$ be (worst-case) number of bit operations performed when multiplying 2 n-bit numbers!
  - $B(n) = 4B(\frac{n}{2}) + \Theta(n)$
- Solution to recurrence?

Recall: (Our) Master Theorem

- Let $T(n)$ be a recurrence relation defined on integers $n \in \mathbb{N}$, having the following form:
  - $T(n) = \theta(1)$ if $n < n_0$
  - $T(n) = aT(n/b) + \theta(n^d)$ if $n \geq n_0$

  a, b, d, n_0 are constants
  $a > 0$
  $b > 1$
  $d \geq 0$
  $n_0 > 0$

- Then
  - $T(n) = \theta(n^d \log n)$ if $d = \log_b a$
  - $T(n) = \theta(n^{\log_b a})$ if $d < \log_b a$
  - $T(n) = \theta(n^d)$ if $d > \log_b a$

$B(n) = 4B(\frac{n}{2}) + \theta(n)$

$a = 4$
$b = 2$
$d = 1$
$\log_b a = 2$ $\quad d = 1$

$\theta(n^2)$

Karatsuba's algorithm

- Recursive algorithm (first try):
  - Break x and y into two halves: x_L and x_R, y_L and y_R
  - Recursively compute the products $x_L y_L$, $x_L y_R$, $x_R y_L$, $x_R y_R$
  - Compute and then return
    $$2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$$
- Let $B(n)$ be (worst-case) number of bit operations
  - $B(n) = 4B(n/2) + \theta(n)$
- Solution to recurrence? $\theta(n^2)$ again!

- Want to compute: $2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$
- Use fact that
  - $x_L y_R + x_R y_L = (x_L + x_R)(y_L + y_R) - x_L y_L - x_R y_R$
- Why is that useful?

- Want to compute: $2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$ x=1
- Use fact that
  - $x_L y_R + x_R y_L = (x_L + x_R)(y_L + y_R) - x_L y_L - x_R y_R$
- Why is that useful?
  - Recursively compute 3 products instead of 4

    $x_L y_L$, $x_R y_R$, $(x_L + x_R)(y_L + y_R)$
  - Compute $x_L y_R + x_R y_L$ with above formula using results

Karatsuba's algorithm

- Multiply(x, y) // Karatsuba's algorithm for multiplying two n-bit integers, x and y
  - if n==1
    - if $x = 0$ or $y = 0$, return 0, else return 1
  - Break x and y into two halves: x_L and x_R, y_L and y_R
  - $P_1 = \text{multiply}(x_L, y_L)$, $P_2 = \text{multiply}(x_R, y_R)$
  - $w = (x_L + x_R)$ and $z = (y_L + y_R)$
  - $P_3 = \text{multiply}(w, z)$
  - Compute and return
    $2^n P_1 + 2^{n/2}(P_3 - P_1 - P_2) + P_2$ //use left shift to multiply by $2^{n/2}$ and 2^n
- Recurrence for number of bit operations $B(n)$?

$3B(\frac{n}{2}) + \Theta(n)$

- Solution to recurrence?

$\Theta(n^{\log_3 2})$

$+\begin{array}{r} 10 \\ 11 \\ \hline \end{array}$ $\begin{array}{r} 11 \\ 11 \\ \hline \end{array}$ $n_2 = 2$

#. $a = 3$, $b = 2$, $d = 1$ $\log_b a = \log_2 3$, $d = 1$

Karatsuba's algorithm

- Karatsuba's algorithm
  - Break x and y into two halves $x_L x_R$, $y_L y_R$
  - Recursively compute the products $x_L y_L$, $x_R y_R$
  - Compute $(x_L + x_R)$ and $(y_L + y_R)$
  - Recursively compute the product $(x_L + x_R)(y_L + y_R)$
  - Compute
    $$2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$$
- Recurrence for number of bit operations $B(n)$?
  - $B(n) = 3B(n/2) + \theta(n)$
- Solution to recurrence?
  - $B(n) = \theta(n^{\log_2 3})$ and $\log_2 3 \approx 1.59$ $\theta(n^{1.59})$

$$\sum_{i=1}^{\infty} \frac{i}{2^i}$$

- Value of $\sum_{i=1}^{k} \frac{i}{2^i}$ when k is a positive integer?
  - Not a geometric series
- We will show that $\sum_{i=1}^{k} \frac{i}{2^i} \leq 2$ for all positive integers k
- Consider $\sum_{i=1}^{k} \frac{i}{2^i} = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \ldots + \frac{k}{2^k}$
  - e.g. for $k = 5$: $\sum_{i=1}^{k} \frac{i}{2^i} = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \frac{5}{32} =$

$$(\frac{1}{2}+\frac{1}{4}+\frac{1}{8}+\frac{1}{16}+\frac{1}{32})+(\frac{1}{4}+\frac{1}{8}+\frac{1}{16}+\frac{1}{32})+(\frac{1}{8}+\frac{1}{16}+\frac{1}{32})+(\frac{1}{16}+\frac{1}{32})+(\frac{1}{32})$$

$$\leq 1 \quad \leq \frac{1}{2} \quad \leq \frac{1}{4} \quad \leq \frac{1}{8} \quad \leq \frac{1}{16}$$

$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} \leq 1 + \underbrace{(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16})}_{\leq 1}$$

$$\leq 2$$

- Here's a proof that $\sum_{i=1}^{k} \frac{i}{2^i} \leq 2$ for all positive integers k
- $\sum_{i=1}^{k} \frac{i}{2^i} = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \ldots + \frac{k}{2^k} \leq 2$
- Can show this is at most 2
  - e.g., $\frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \frac{5}{32} =$
  - $(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32}) + (\frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32}) + (\frac{1}{8} + \frac{1}{16} + \frac{1}{32}) + (\frac{1}{16} + \frac{1}{32}) + (\frac{1}{32})$
- More generally:
  - Let $S_d = \sum_{i=d}^{k} \frac{1}{2^i}$
  - Then $\sum_{i=1}^{k} \frac{i}{2^i} = S_1 + S_2 + \ldots + S_k$
  - $S_1 \leq 1$ and $S_d = S_{d-1} - \frac{1}{2^{d-1}}$ for $d > 1$
  - By induction can show $S_d \leq \frac{1}{2^{d-1}}$ for all d so $\sum_{i=1}^{k} \frac{i}{2^i} = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \ldots \frac{1}{2^k} < 2$

- $\sum_{i=1}^{\infty} \frac{i}{2^i}$ = ??? (infinite series)
- $\sum_{i=1}^{\infty} \frac{i}{2^i} = 2$ because it's equal to expected number of tosses of a fair coin needed to get heads, which is 2
- Proof that $\sum_{i=1}^{\infty} \frac{i}{2^i} = 2$
  - Consider the (infinite) geometric series: $\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$ if $|x| < 1$
  - Differentiate both sides wrt x, get
    $$\sum_{i=0}^{\infty} i x^{i-1} = \frac{1}{(1-x)^2}$$
    Then multiply both sides by x, get
    $$\sum_{i=0}^{\infty} i x^i = \frac{x}{(1-x)^2}$$
  - Plug in $x = \frac{1}{2}$ get $\sum_{i=1}^{\infty} \frac{i}{2^i} = 2$

Keep tossing fair coin until get heads

Expected # flips when see first head

$$= \sum_{i=1}^{\infty} \text{Prob}[\text{See first head on flip } i] * i$$

TTTT...TH

i-1

$$= \sum_{i=1}^{\infty} \left(\frac{1}{2^i}\right) * i = \sum_{i=1}^{\infty} \frac{i}{2^i}$$

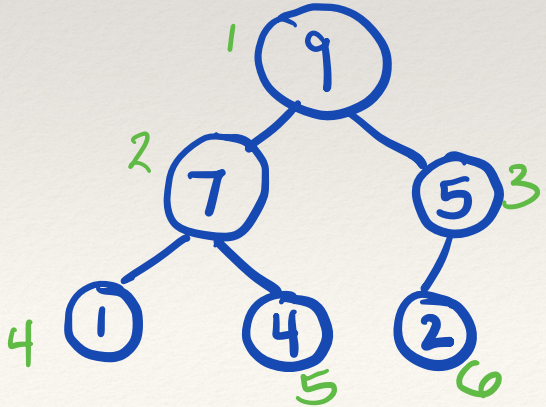
Further Sorting + Order Statistics

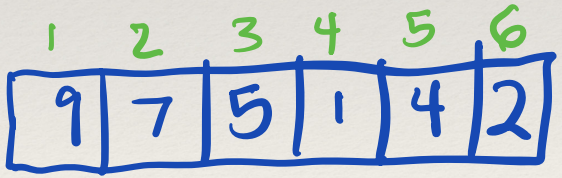
Heapsort

- Good data structures can lead to efficient algorithms

Heap

- (Array implementation)
- Represented by array A[1…n] that corresponds to binary tree
- Associated variable A.heap-size stores the number of elements in the heap
- Only elements in first A.heap-size entries are in the heap (remaining elements are garbage)
- Corresponding tree is the one you'd get by adding the A.heap-size entries to a binary tree, level by level, from left to right
- Associate each node of tree with corresponding array index





- Can easily compute array index for parent and children of node i
  - Parent(i): return $\lfloor i/2 \rfloor$
  - Left-child(i): return 2i
  - Right-child(i): return 2i+1

Node 5 Parent $\lfloor \frac{5}{2} \rfloor = 2$

Node 2 Left child $2 \cdot 2 = 4$

- Max-heap
  - A[parent(i)] >= A[i]
  - Max element is at root
- Min-heap
  - A[parent(i)] <= A[i]
  - Min element is at root
- Heap-sort idea
  - Put elements of the array into max-heap order.
  - Repeat
    - Delete max element from heap and output.
    - Repair heap
  - (This outputs elements in reverse sorted order)
- Before showing implementation of Heapsort, need to introduce heap operations

Relevant Heap Operations

- (For Max-Heap)
  - Max-Heapify(A,i) // see next slide
  - Max-Heap-Extract-Max(A) // remove max element from heap and repair
  - BuildMaxHeap(A,n) // take array A of size n and turn it into max-heap

Max-Heapify(A,i)

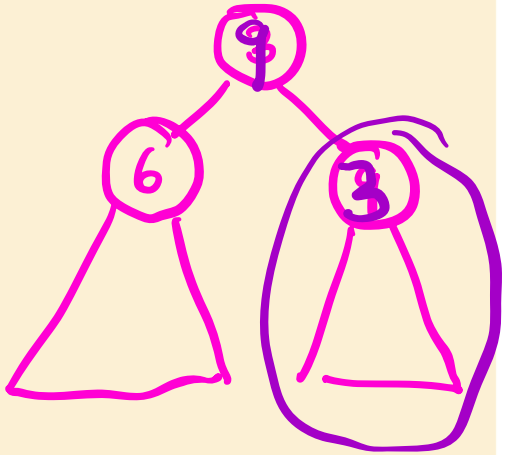
- Assumes that the left and right subtrees of node i are max-heaps
  - but tree rooted at node i might not be a max-heap
    - A[i] might be less than A[left(i)] and A[right(i)]
- Rearranges elements in tree rooted at node i to turn it into a max-heap
  - How? "Bubble down" (see next slide)

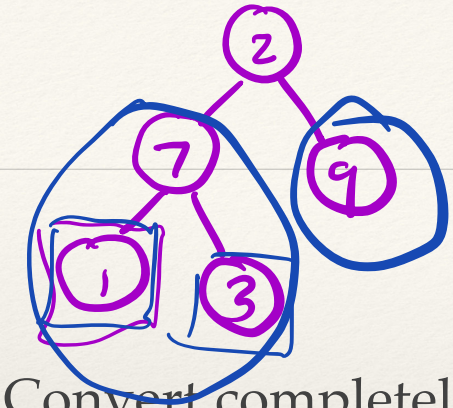
Max-Heapify(A,i)

- Max-Heapify(A,i)
  - Compare A[i] to elements in left and right children of node i (if they exist)
  - If no children, or A[i] greater than or equal to those elements, done
  - else swap elements in A[i] and child with larger element, and recursively call Max-Heapify on that child

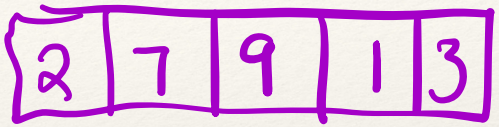
Max-Heapify Pseudocode

```
MAX-HEAPIFY(A, i)
 1  l = LEFT(i)
 2  r = RIGHT(i)
 3  if l ≤ A.heap-size and A[l] > A[i]
 4       largest = l
 5  else largest = i
 6  if r ≤ A.heap-size and A[r] > A[largest]
 7       largest = r
 8  if largest ≠ i
 9       exchange A[i] with A[largest]
10       MAX-HEAPIFY(A, largest)
```

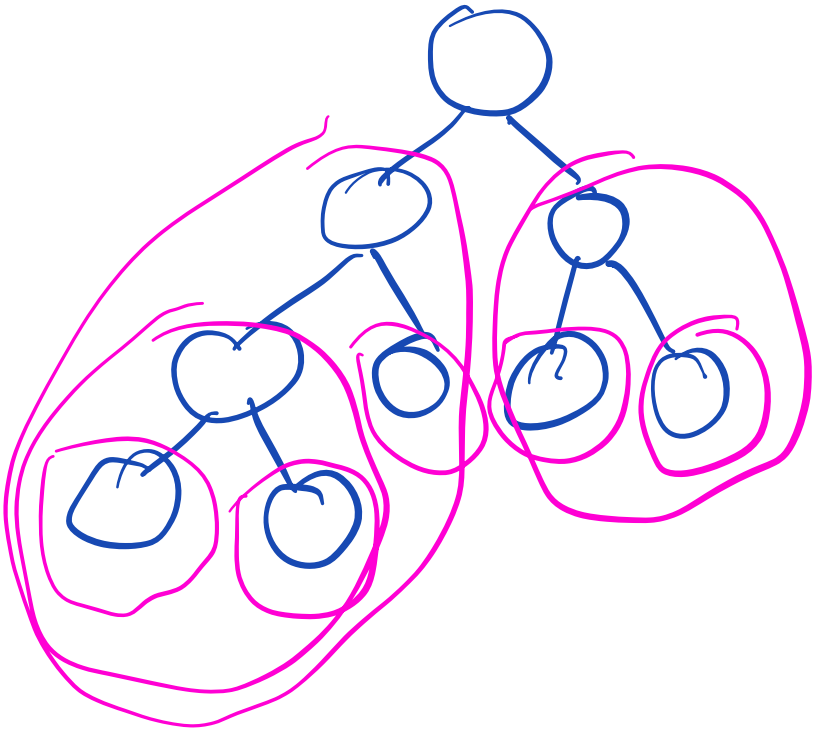




Build-Max-Heap

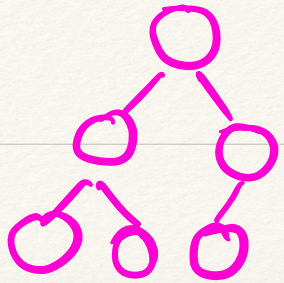


- Convert completely unordered array A into a max-heap
  - Beginning at bottom level of tree, and moving up level by level:
    - Run Max-Heapify on all nodes on current level, so all subtrees rooted at that level are max-heaps.
  - Can actually begin at second-to-bottom level (why?)



Pseudocode for Build-Max-Heap

- Build-Max-Heap(A, n)
  - A.heapsize = n
  - for i = $\lfloor n/2 \rfloor$ downto 1
    - Max-Heapify(A,i)



- Running time of Max-Heapify
  - $\theta(h)$ where h is height of subtree with root at node i [height here is max # edges from node to leaf]
  - $= \theta(\log n)$ if subtree has n elements

Pseudocode for Build-Max-Heap

```
Build-Max-Heap(A, n)
    A.heapsize = n
    for i = ⌊n/2⌋ downto 1
        Max-Heapify(A,i)
```

- Correctness of Build-Max-Heap
  - Relies on fact that all descendants of node i have indices higher than i
  - Loop Invariant: At start of each iteration of for-loop, nodes i+1 through n are all roots of Max-Heaps
    - Base case: Property holds for i=$\lfloor n/2 \rfloor$ because nodes $\lfloor n/2 \rfloor + 1, \ldots, n$ are all leaf nodes
    - Maintenance: Assuming property holds before an iteration, children of node i are both roots of Max-Heaps. So calling Max-Heapify(A,i) makes node i the root of a Max-Heap, values not in that Max-Heap remain unaffected.
      So before next iteration, nodes $\lfloor n/2 \rfloor, \ldots, n$ are all roots of Max-Heaps.
    - Termination: Clear it terminates. At end, i=0, so by Loop Invariant, node 1 is root of Max-Heap

Runtime of Build-Max-Heap

- Height of entire tree is
- Build-Max-Heap runs Max-Heapify $\lfloor \frac{n}{2} \rfloor$ times
- Max-Heapify runs in time $\theta(h)$, where h is height of subtree
- So each Max-Heapify runs in time O($\log n$)

  $\Rightarrow$ Build-Max-Heap runs in time O($n \log n$)

Tighter bound on Build-Max-Heap

- Previous analysis overestimated time for many of the calls to Max-Heapify
- More careful analysis:
  - Runtime of Max-Heapify is at most ch for some constant c (since it is $\theta(h)$)
  - Tree has 1 node that is root of subtree of height $\lfloor \log_2 n \rfloor$
    2 nodes that are roots of subtrees of height $\lfloor \log_2 n \rfloor - 1$
    …
    2^i nodes that are roots of subtrees of height
    …
  - $\leq 2^{\lfloor \log_2 n \rfloor}$ nodes that are roots of subtrees of height
  - Total runtime at most: $\sum_{h=0}^{\lfloor \log_2 n \rfloor} 2^{\lfloor \log_2 n \rfloor - h} \cdot ch \leq \sum_{h=0}^{\lfloor \log_2 n \rfloor} 2^{\log_2 n} \cdot \frac{1}{2^h} \cdot ch = \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^h} \cdot ch \leq cn \sum_{h=0}^{\infty} \frac{h}{2^h} = 2cn$ WHY?

- Runtime of Build-MaxHeap is $\theta(n)$

Heapsort

```
HEAPSORT(A, n)
1  BUILD-MAX-HEAP(A, n)
2  for i = n downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size − 1
5      MAX-HEAPIFY(A, 1)
```

Loop invariant:

Heap only contains A[1…i]. They are the smallest items in A. Remaining elements are in A[i+1…n] in sorted order

Heapsort

```
HEAPSORT(A, n)
1  BUILD-MAX-HEAP(A, n)
2  for i = n downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size − 1
5      MAX-HEAPIFY(A, 1)
```

Runtime?